

---

```

clc; clearvars; close all

addpath('src');

%{
Would make more sense for it to start on the ground then make its way up.
That way, whenever it reaches a local maxima on diameter, it'll just stop

Also, can add feature where if they run into each other they'll know
they've gone over the object.
%}

lh_grip = gripper('basePosition', [-3.5 2.5], 'parallelLinkLength', 1,
'endEffectorLength', 4);
rh_grip = gripper('handedness', 'r', 'basePosition', [4 2.5],
'parallelLinkLength', 1, 'endEffectorLength', 4);

```

## Simulataneous Grip

```

searchParams = getSearchParams();
hand = gripperSet(lh_grip, rh_grip, searchParams);

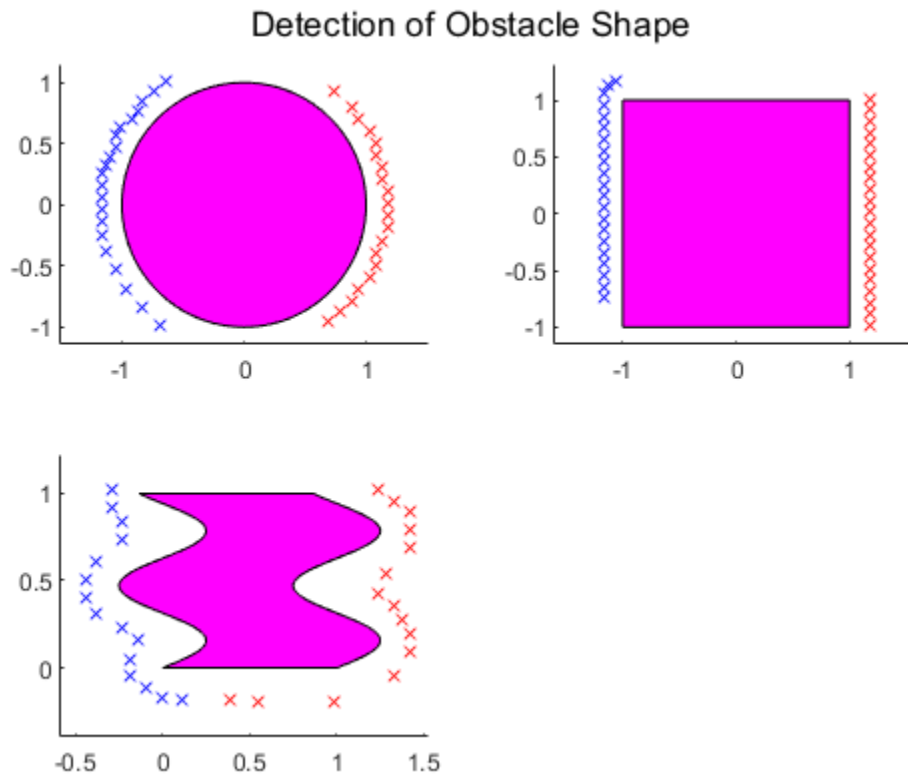
% ** Multiple shapes **
shapes = ["circle", "square", "sine"];
figure();
sgtitle("Detection of Obstacle Shape");
for i = 1:numel(shapes)

    obst = obstacle("shape", shapes(i));
    subplot(2, 2, i)
    hold on
    hand.predictObstacle(obst);
    obst.drawObstacle;
    hold off
    axis padded
end

%** Single obstacle **
% obst = obstacle("shape", "square");

% hand.detectObstacle(obst, searchParams);
% hand.predictObstacle(obst, searchParams);

```



## Publishing

```
function searchParams = getSearchParams

    searchParams.startY = 2;
    searchParams.endY = 0;
    searchParams.startTheta = pi/4;
    searchParams.endTheta = 3*pi/4;
    searchParams.dy = .1;
    searchParams.dtheta = .05;

    searchParams.animateBool = 0;
    searchParams.saveBool = 0;

    if searchParams.saveBool
        searchParams.animateBool = 1;
    end
end

classdef gripper
    %UNTITLED2 Summary of this class goes here
    % Detailed explanation goes here
end
```

---

```

properties
    baseLinkLength = 1;
    parallelLinkLength = 3;
    endEffectorLength = 5;

    basePosition = [0,0];
    handedness = "l"
end

methods
    function obj = gripper(varargin)

        p = inputParser();
        addParameter(p, 'baseLinkLength', obj.baseLinkLength);
        addParameter(p, 'parallelLinkLength', obj.parallelLinkLength);
        addParameter(p, 'endEffectorLength', obj.endEffectorLength);
        addParameter(p, 'basePosition', obj.basePosition);
        addParameter(p, 'handedness', obj.handedness);

        parse(p, varargin{:});

        obj.baseLinkLength = p.Results.baseLinkLength;
        obj.parallelLinkLength = p.Results.parallelLinkLength;
        obj.endEffectorLength = p.Results.endEffectorLength;
        obj.basePosition = p.Results.basePosition;
        obj.handedness = p.Results.handedness;

    end

    function [positions, tip] = definePositions(obj,theta)
        % o: origin; b: base; p = outside parallel; e: end effector
        r_ob1 = obj.basePosition;
        r_b1b2 = [-obj.baseLinkLength, 0];
        r_b2p = [-obj.parallelLinkLength*cos(theta),
obj.parallelLinkLength*sin(theta)];
        r_pe = [obj.endEffectorLength, 0];

        if obj.handedness == 'r'
            r_b1b2 = -r_b1b2;
            r_b2p(1) = -r_b2p(1);
            r_pe = -r_pe;
        end

        b1 = r_ob1;
        b2 = b1 + r_b1b2;
        p = b2 + r_b2p;
        e = p + r_pe;

        positions = [b1; b2; p; e];

        % Tip

```

---

---

```

    thetas = linspace(-pi/2, pi/2, 200);
    r = .2;
    tip_x = r*cos(thetas);
    tip_y = r*sin(thetas);

    if obj.handedness == 'r'
        tip_x = -tip_x;
    end

    tip = [tip_x; tip_y] + e';
end

function handles = plotPosition(obj, theta)
    eps = 0.1;
    [positions, tip] = obj.definePositions(theta);
    % base link
    b1x = positions(1, 1);
    b1y = positions(1, 2);
    b2x = positions(2, 1);
    b2y = positions(2, 2);

    base_x = [b1x, b1x, b2x, b2x];
    base_y = [b1y + eps, b2y - eps, b2y - eps, b1y + eps];

    % Outside Parallel Linkage
    p2x = positions(3, 1);
    p2y = positions(3, 2);

    par_x = [p2x, b2x];
    par_y = [p2y, b2y];

    % End effector
    ex = positions(4, 1);
    ey = positions(4, 2);

    e_x = [p2x, p2x, ex, ex];
    e_y = [p2y-eps, ey+eps, ey+eps, p2y-eps];

    tip_x = tip(1, :);
    tip_y = tip(2, :);

    gca;
    hold on
    h1 = fill(base_x, base_y, 'k');
    h2 = plot(par_x, par_y, 'LineWidth', 2, 'Color', 'r');
    if obj.handedness == 'l'
        h3 = plot(par_x + obj.baseLinkLength, par_y, 'LineWidth', 2,
'Color', 'b');
    else
        h3 = plot(par_x - obj.baseLinkLength, par_y, 'LineWidth', 2,
'Color', 'b');
    end
    h4 = fill(e_x, e_y, 'r');
    h5 = fill(tip_x, tip_y, 'k');

```

---

---

```

        axis([-7 7 -5 5]);

        handles = [h1, h2, h3, h4, h5];
    end

    function animateGripper(obj, thetas)
        gca;

        for i = 1:length(thetas)
            theta = thetas(i);
            hs = obj.plotPosition(theta);
            drawnow
            delete(hs);
        end
    end

    function contactPositions = detectObstacle(obj, obstacle,
searchParams)
        obstacle.drawObstacle();

        startY = searchParams.startY;
        endY = searchParams.endY;
        startTheta = searchParams.startTheta;
        endTheta = searchParams.endTheta;
        dy = searchParams.dy;
        dtheta = searchParams.dtheta;

        searchYs = startY:-dy:endY;
        searchThetas = startTheta:dtheta:endTheta;

        contactPositions = [];

        for i = 1:numel(searchYs)
            obj.basePosition(2) = searchYs(i);
            ctr = 1;
            contact = 0;
            while ~contact && ctr <= numel(searchThetas)
                cur_theta = searchThetas(ctr);
                [positions,tip] = obj.definePositions(cur_theta);
                endEffectorPosition = positions(end, :);
                [in, on] = inpolygon(tip(1,:), tip(2,:),
obstacle.coords(1,:), obstacle.coords(2,:));
                if (sum(in) + sum(on)) > 0
                    contact = 1;
                    contactPositions =
[contactPositions;endEffectorPosition];
                end
                ctr = ctr + 1;
            if searchParams.animateBool
                hs = obj.plotPosition(cur_theta);
                drawnow;
                pause(.2);
            end
        end
    end
end

```

---

---

```

        delete(hs);
    end
end

end
end

function predictObstacle(obj, obstacle, searchParams)
    endEffectorPositions = obj.detectObstacle(obstacle,
searchParams);
    x_vals = endEffectorPositions(:, 1);
    y_vals = endEffectorPositions(:, 2);
    gca;
    if obj.handedness == 'r'
        color = 'r';
    else
        color = 'b';
    end
    scatter(x_vals, y_vals, 'x', color);
    axis equal
end

end
end

classdef gripperSet

    properties
        lhs
        rhs
        sP = [];
    end

    methods
        %** Constructor
        function obj = gripperSet(lhs, rhs,searchParams)
            obj.lhs = lhs;
            obj.rhs = rhs;
            if nargin == 3
                obj.sP = searchParams;
            end
        end

        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
        %***** 'Main' Functions for Obstacle Detection *****
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        %** Search for contact with the obstacle at a given height
        function [contactPosition_lhs, contactPosition_rhs] =
searchHeight(obj, obstacle, height)

            obstacle.drawObstacle();

```

---

---

```

searchThetas = obj.sP.startTheta:obj.sP.dtheta:obj.sP.endTheta;

contactPosition_lhs = [];
contactPosition_rhs = [];

obj.lhs.basePosition(2) = height;
obj.rhs.basePosition(2) = height;

ctr = 1;

contact_lhs = 0;
contact_rhs = 0;

% Both sides same time
while ~(contact_lhs && contact_rhs) && (ctr <=
numel(searchThetas))
    % current angle
    cur_theta = searchThetas(ctr);

    % Define positions of each mechanism if they haven't
    % made contact yet
    if ~contact_lhs
        [contact_lhs, contact_lhs_theta, contactPosition_lhs] =
detectContact(obj, cur_theta, 'l', obstacle);
    end

    if ~contact_rhs
        [contact_rhs, contact_rhs_theta, contactPosition_rhs] =
detectContact(obj, cur_theta, 'r', obstacle);
    end

    ctr = ctr + 1;

    % This will freeze the arm in the same position if it has
    % already come into contact with the obstacle
    if obj.sP.animateBool
        obj.animateSet(cur_theta, contact_lhs, contact_rhs,
contact_lhs_theta, contact_rhs_theta)
    end
end
end

%** Search for contact points with the obstacle at a RANGE of
%heights
function [contactPositions_lhs, contactPositions_rhs] =
detectObstacle(obj, obstacle)

searchYs = obj.sP.startY:-obj.sP.dy:obj.sP.endY;

contactPositions_lhs = [];
contactPositions_rhs = [];

```

---

---

```

    if obj.sP.saveBool
        writerObj = VideoWriter('animation.mp4');
        writerObj.FrameRate = 30;
        open(writerObj);
    end

    for i = 1:numel(searchYs)
        [l, r] = searchHeight(obj, obstacle, searchYs(i));
        contactPositions_lhs = [contactPositions_lhs; l];
        contactPositions_rhs = [contactPositions_rhs; r];
    end

end

*** Take the search data obtained from probing the whole obstacle
% and plot the points where contact was made
function predictObstacle(obj, obstacle)
    [endEffectorPositions_lhs, endEffectorPositions_rhs] =
obj.detectObstacle(obstacle);

    x_vals_lhs = endEffectorPositions_lhs(:, 1);
    y_vals_lhs = endEffectorPositions_lhs(:, 2);
    x_vals_rhs = endEffectorPositions_rhs(:, 1);
    y_vals_rhs = endEffectorPositions_rhs(:, 2);

    gca;
    hold on
    scatter(x_vals_lhs, y_vals_lhs, 'x', 'b');
    scatter(x_vals_rhs, y_vals_rhs, 'x', 'r');
    axis equal
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
***** 'Helper' Functions *****
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

*** detect contact between the hand and the obstacle
function [contact, contact_theta, contactPosition] =
detectContact(obj, theta, hand, obstacle)

    contact = 0;
    contact_theta = [];
    contactPosition = [];

    if hand == 'r'
        [positions, tip] = obj.rhs.definePositions(theta);
    elseif hand == 'l'
        [positions, tip] = obj.lhs.definePositions(theta);
    end

    % Check for collisions and return contact, theta, and position

```

---



---

```

        % if so
        endEffectorPosition = positions(end, :);
        [in, on] = inpolygon(tip(1,:), tip(2,:), obstacle.coords(1,:),
obstacle.coords(2,:));
        if sum([in, on]) > 0
            contact= 1;
            contact_theta = theta;
            contactPosition = endEffectorPosition;
        end

    end

    %** Plot the grippers, keeping them steady if in contact
    function animateSet(obj, cur_theta, contact_lhs, contact_rhs,
contact_lhs_theta, contact_rhs_theta)
        if ~contact_lhs
            h_lhs = obj.lhs.plotPosition(cur_theta);
        else
            h_lhs = obj.lhs.plotPosition(contact_lhs_theta);
        end

        if ~contact_rhs
            h_rhs = obj.rhs.plotPosition(cur_theta);
        else
            h_rhs = obj.rhs.plotPosition(contact_rhs_theta);
        end

        drawnow;
        if obj.sP.saveBool
            writeVideo(writerObj, getframe(gcf));
        end
        delete([h_lhs,h_rhs]);

    end

end

end

classdef obstacle

    properties
        shape = 'square'
        coords
    end

    methods
        function obj = obstacle(varargin)
            p = inputParser();
            addParameter(p, "shape", obj.shape);

            parse(p, varargin{:});

```

---

---

```

        obj.shape = p.Results.shape;
        obj = obj.makeCoords();
    end

    function obj = makeCoords(obj)

        if obj.shape == "square"
            obj.coords = [-1 -1 1 1; -1 1 1 -1];
        elseif obj.shape == "circle"
            thetas = linspace(0, 2*pi, 500);
            r = 1;
            obj.coords = [r*cos(thetas); r*sin(thetas)];
        elseif obj.shape == "sine"
            y_vals = linspace(0, 1, 100);
            x_vals = .25*sin(10*y_vals);

            obj.coords = [x_vals, flip(x_vals) + 1; y_vals,
flip(y_vals)];
        end

    end

    function drawObstacle(obj)
        % obj.makeCoords();
        gca;
        fill(obj.coords(1, :), obj.coords(2, :), 'm');

    end

end
end
end

```

*Published with MATLAB® R2024b*